

Practical 1

Introduction to React and Component Architecture

Vite + React Student Portfolio with Reusable Component Architecture and Props

Student Name:	Mantra Patel
Student ID:	24DCS076
Course:	Advanced Web Development Frameworks (ITUE301)
Institute:	DEPSTAR - CHARUSAT
Branch / Sem:	B.Tech CSE - Semester 5
Academic Year:	2025-2026
Dev Server Port:	http://localhost:5173

Objective: Set up a modern Vite + React application and construct a student developer portfolio using independently structured, reusable functional components communicating through props.

1. Theory & Concepts

React is a declarative JavaScript library for building user interfaces through a component-based architecture. Components are self-contained, reusable units that accept inputs (props) and return JSX describing what the UI should render.

Key concepts demonstrated in Practical 1:

- Functional Components: Declared with plain JavaScript functions; return JSX markup.
- JSX (JavaScript XML): HTML-like syntax embedded in JavaScript, compiled by Vite's Babel/SWC transform.
- Props (Properties): Read-only data passed from a parent component to its children. Enforces a single source of truth.
- Dynamic Rendering: Using `Array.prototype.map()` with unique key props to eliminate repeated JSX.
- Single Responsibility Principle: Each component does exactly one job (Header displays hero, Skills shows tags, etc.).
- Scroll-based Active Section: `useEffect` registers a scroll listener that updates `activeSection` state, highlighting the correct NavBar link.

2. Component Architecture

Component Tree

```
App.jsx (root - declares all data)
|-- NavBar.jsx    (scroll-based active section highlight)
|-- Header.jsx    (name prop, themeColor prop, role prop)
|-- About.jsx     (bio prop)
|-- Skills.jsx    (skillList array prop -> dynamic .map())
|-- Projects.jsx  (projects array prop -> 3 cards)
`-- Footer.jsx    (name prop)
```

3. Key Code Snippets

Skills.jsx - Props Array & Dynamic Rendering

```
// Skills.jsx - dynamic rendering from prop array
function Skills({ skillList = [] }) {
  return (
    <section className='skills'>
      <p className='section-heading'>Skills</p>
      <div className='skill-tags'>
        {skillList.map((skill) => (
          <span className='tag' key={skill}>{skill}</span>
        ))}
      </div>
    </section>
  )
}
```

```

    </section>
  )
}

```

Header.jsx - Props with Inline Dynamic Styling

```

// Header.jsx - name and themeColor passed as props
function Header({ name, themeColor = '#3b82f6', role }) {
  return (
    <header className='hero'>
      <h1 className='hero-name' style={{ color: themeColor }}>
        {name}
      </h1>
      <p className='hero-role'>{role}</p>
    </header>
  )
}

```

App.jsx - Props Declaration & Passing

```

// App.jsx - single source of truth for all data
const studentName = 'Mantra Patel'
const themeColor = '#38bdf8'
const technicalSkills = ['HTML5', 'CSS3', 'React.js', 'Node.js', ...]

<Header name={studentName} themeColor={themeColor} />
<Skills skillList={technicalSkills} />

```

4. Application Screenshots

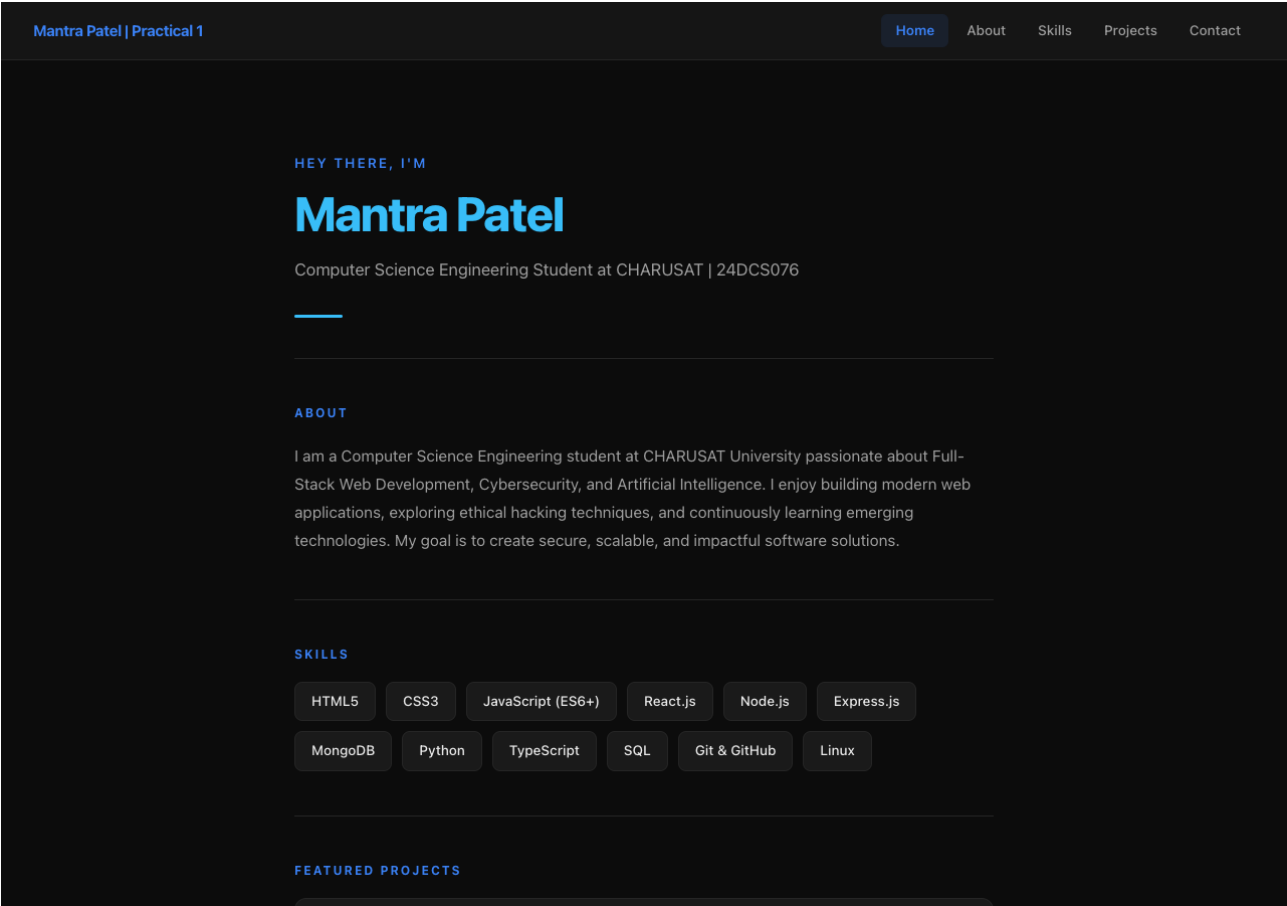


Figure: Practical 1 - Hero Section, About, and Skills tags (Dark Theme)

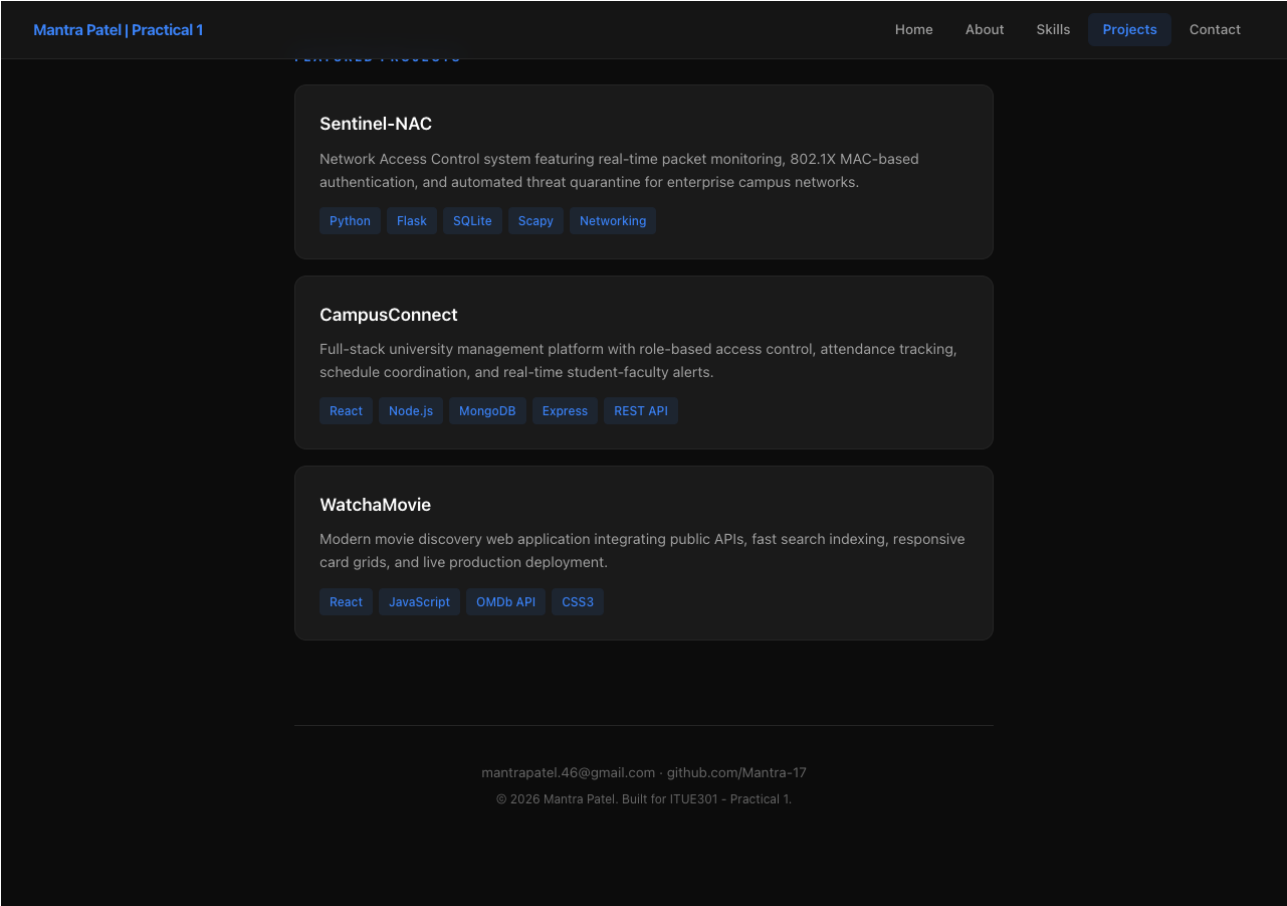


Figure: Practical 1 - Projects Section and Footer

5. Testing Checklist

- npm install runs without errors in Practical-1/
- npm run dev launches on http://localhost:5173 with strictPort: true
- All 6 components render: NavBar, Header, About, Skills, Projects, Footer
- Props verified: Changing studentName in App.jsx immediately re-renders Header
- Skills array of 12 items rendered dynamically via .map() with unique keys
- Projects component displays 3 cards with title, description, tech badges
- NavBar active section updates correctly as user scrolls through each section
- No console errors or React warnings in browser DevTools
- Production build (npm run build) succeeds with 0 TypeScript/Vite errors

6. Learning Outcomes

After completing Practical 1, I can:

- Configure a modern React development environment using Vite with HMR and ESLint.
- Decompose a UI into small, single-responsibility functional components.
- Pass data from parent to child components using props without breaking component encapsulation.
- Render lists dynamically using Array.map() with proper React key assignment.
- Apply inline dynamic styling through props (themeColor -> style={{ color: themeColor }}).
- Implement a scroll-aware NavBar using useEffect and addEventListener without memory leaks.